Part 1

Do some investigation into the Swing framework and write a short paragraph describing the purpose of the Swing framework. Submit a class diagram of the components of Swing.

Swing is a graphical user interface (GUI) framework in Java that is used to build desktop applications/programs and does so through a series of components and tools, such as windows, buttons, tables, menus, etc.

It was developed based on the *MVC (Model-View-Controller)* architectural design pattern and modified for its purposes called a *separable model architecture*.

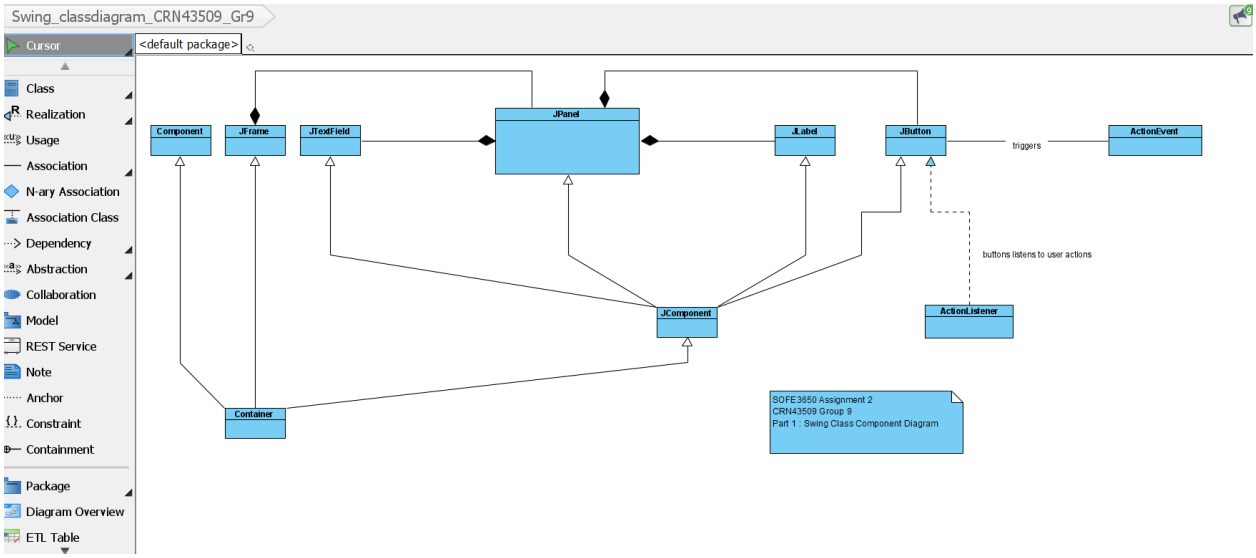
- Model: Data/Logic of program
- View: User Interface (what user interacts with when using a program/app, such as windows, buttons, microphone, etc.)
- Controller: connection between the model and the view; taking input from the user and converting it into instructions for the Model to carry out.

Swing framework combines the *View* and *Controller* entities into one object called a *UI (user interface) delegate*, separating it from the *Model*, which makes it easier to organize and maintain the code/program and lowers the execution time of code.

It uses an *event-driven model*, only responding to a user's input/interactions, and also allows the user to change the look and feel of the GUI at any time without affecting the model/data.

- *JFrame* : top-level window that contains the application
- Panels : organize layout in frame (*JPanel*)
- Components : interactive parts of the interface (*JButton*, *JLabel*, *JTable*, *etc*.)
- Listener : object that takes input (*listens*) and responds with output for various events in the GUI, such as keyboard typing (*KeyListener*), mouse clicking (*MouseListener*), navigating application, etc.

It is also written with *lightweight components* - completely in Java - and so is not restricted to any native operating system (OS) and can be used without error on any platform or OS.



Part 2

How the template implements MVC:

- Model: a simple data holder (in the starter template it stores a person's first/last name).
- View: a Swing JFrame with text fields and buttons (UI only).
- Controller: registers action listeners on the view's buttons, reads values from the view, writes them into the model, and shows dialogs.

  The assignment explicitly states the template is a classical MVC model that reflects the Java Swing framework with files named Model, View, Controller, and that the demo UI "takes as input a person's name and will display it back again."

How it differs from a strict/conventional MVC (as taught in lectures):

- No observer/notification from Model to View: In strict MVC the View typically observes the Model (e.g., via listeners / property change events) so that UI updates when data

changes. The template instead has the Controller push updates directly to the View (no observer pattern / data binding).

- View exposes widgets via getters: This is common in Swing examples, but it couples the Controller to concrete Swing components. A stricter MVC would keep the controller UI-agnostic (working against an interface) and have the view implement that interface.

- Controller performs UI concerns (dialogs): In pure MVC, user messaging could be considered a View concern. Here, the Controller also triggers JOptionPane, so UI concerns leak into the Controller.

- “Passive View” vs “Supervising Controller”: The template is closer to a Passive View: View is mostly dumb; Controller reads/writes the Model and tells the View what to render, rather than the View binding directly to the Model.

Part 3

How it differs from a strict/conventional MVC (as taught in lectures):

Model.java:

```
import java.io.*;
import java.math.BigDecimal;
import java.util.*;

public class Model {
   private final Map<String, Product> catalog = new LinkedHashMap<>();
   private final List<Product> scanned = new ArrayList<>();
   private java.math.BigDecimal subtotal = java.math.BigDecimal.ZERO;

   public void loadCatalog(String pathOrResource) {
       List<String> lines = readAllLines(pathOrResource);
       for (String line : lines) {
```

```
            line = line.trim();
            if (line.isEmpty() || line.startsWith("#")) continue;

            // Before splitting, normalize: join "$ <num>" into "$<num>"
            line = line.replaceAll("\\$\\s+(\\d)", "\\$$1");


            String[] parts = line.split("\\s+");
            if (parts.length < 3) continue;

            String upc = parts[0];
            String rawPrice = parts[parts.length - 1].replace("$", "").replace(",",
"");
            BigDecimal price = new BigDecimal(rawPrice);
            String name = String.join(" ", Arrays.asList(parts).subList(1, parts.length
- 1));

            Product p = new Product(upc, name, price);
            catalog.put(upc, p);
        }
        if (catalog.isEmpty()) throw new IllegalStateException("No products loaded from
" + pathOrResource);
    }

    private List<String> readAllLines(String pathOrResource) {
        // already implemented: tries filesystem first, then classpath "/products.txt"
        // This commit documents that behavior; no code change if your current method
matches.

        File f = new File(pathOrResource);
        try {
            if (f.exists()) {
                return java.nio.file.Files.readAllLines(f.toPath());
            } else {
                try (InputStream in = getClass().getResourceAsStream("/" +
pathOrResource)) {
                    if (in == null) throw new FileNotFoundException(pathOrResource);
                    try (BufferedReader br = new BufferedReader(new
InputStreamReader(in))) {
                        List<String> out = new ArrayList<>();
                        String s;
                        while ((s = br.readLine()) != null) out.add(s);
```

```
                        return out;
                    }
                }
            }
        } catch (Exception e) {
            throw new RuntimeException("Failed to read " + pathOrResource + ": " +
e.getMessage(), e);
        }
    }

    public Collection<Product> getCatalog() { return
Collections.unmodifiableCollection(catalog.values()); }
    public Optional<Product> findByUpc(String upc) { return
Optional.ofNullable(catalog.get(upc)); }

    public void addItemByUpc(String upc) {
        Product p = findByUpc(upc).orElseThrow(() ->
            new IllegalArgumentException("UPC not found in catalog: " + upc));
        addItem(p);
    }


    public void addItem(Product p) {
        scanned.add(p);
        subtotal = subtotal.add(p.getPrice());
    }

    public List<Product> getScannedItems() { return
Collections.unmodifiableList(scanned); }
    public java.math.BigDecimal getSubtotal() { return subtotal; }
    public void clear() { scanned.clear(); subtotal = java.math.BigDecimal.ZERO; }
}
```

Controller.java:

```
import javax.swing.*;
import java.math.BigDecimal;
import java.text.NumberFormat;
import java.util.Locale;
```

```
public class Controller {
   private final Model model;
   private final View view;
   private final Scanner scanner;
   private final NumberFormat currency =
NumberFormat.getCurrencyInstance(Locale.CANADA);

   public Controller(Model model, View view, Scanner scanner) {
       this.model = model;
       this.view = view;
       this.scanner = scanner;
       attachHandlers();
       refreshTotals();
   }

   private void attachHandlers() {
       // Buttons
       view.getScanButton().addActionListener(e -> onScan());
       view.getClearButton().addActionListener(e -> onClear());
       view.getAddByUpcButton().addActionListener(e -> onAddByUpc());

       // Keyboard shortcuts: Enter = Scan, Ctrl+L = Clear
       JRootPane root = view.getRootPane();
       root.setDefaultButton(view.getScanButton());
       InputMap im = root.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW);
       ActionMap am = root.getActionMap();
       im.put(KeyStroke.getKeyStroke("control L"), "clear");
       am.put("clear", new AbstractAction() {
           @Override public void actionPerformed(java.awt.event.ActionEvent e) {
onClear(); }
       });
   }

   private void onScan() {
       try {
           String upc = scanner.pickRandomUpc();
           model.addItemByUpc(upc);
           Product p = model.findByUpc(upc).get();
           view.addItemLine(formatItem(p));
           refreshTotals();
       } catch (Exception ex) {
           JOptionPane.showMessageDialog(view, "Scan failed: " + ex.getMessage(),
```

```
                    "Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    private void onAddByUpc() {
        try {
            String upc = view.getEnteredUpc();
            if (upc.isEmpty()) return;
            model.addItemByUpc(upc);
            Product p = model.findByUpc(upc).get();
            view.addItemLine(formatItem(p));
            refreshTotals();
            view.clearUpcField();
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(view, "Add by UPC failed: " +
ex.getMessage(),
                    "Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    private void onClear() {
        if (model.getScannedItems().size() >= 5) {
            int res = JOptionPane.showConfirmDialog(view,
                    "Clear all scanned items?", "Confirm",
JOptionPane.OK_CANCEL_OPTION);
            if (res != JOptionPane.OK_OPTION) return;
        }
        model.clear();
        view.resetList();
        refreshTotals();
    }

    private void refreshTotals() {
        BigDecimal subtotal = model.getSubtotal();
        view.setSubtotalText("Subtotal: " + currency.format(subtotal));
    }

    private String formatItem(Product p) {
        return p.getName() + " — " + currency.format(p.getPrice()) + "  [" + p.getUpc()
+ "]";
    }
```

```
    public void show() {
        SwingUtilities.invokeLater(() -> view.setVisible(true));
    }
}
```

View.java:

```
import javax.swing.*;
import java.awt.*;

public class View extends JFrame {
    private final DefaultListModel<String> listModel = new DefaultListModel<>();
    private final JList<String> itemList = new JList<>(listModel);
    private final JLabel subtotalLabel = new JLabel("Subtotal: $0.00");
    private final JButton scanButton = new JButton("Scan");
    private final JButton clearButton = new JButton("Clear");

    // new fields
    private final JTextField upcField = new JTextField(10);
    private final JButton addByUpcButton = new JButton("Add by UPC");

    public View() {
        super("Cash Register");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        itemList.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 13));
        itemList.setVisibleRowCount(12);
        JScrollPane scroll = new JScrollPane(itemList);

        JPanel bottom = new JPanel(new BorderLayout());
        bottom.add(subtotalLabel, BorderLayout.WEST);

        JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        actions.add(new JLabel("UPC:"));
        actions.add(upcField);
        actions.add(addByUpcButton);
        actions.add(clearButton);
        actions.add(scanButton);
```

```
        bottom.add(actions, BorderLayout.EAST);

        add(scroll, BorderLayout.CENTER);
        add(bottom, BorderLayout.SOUTH);

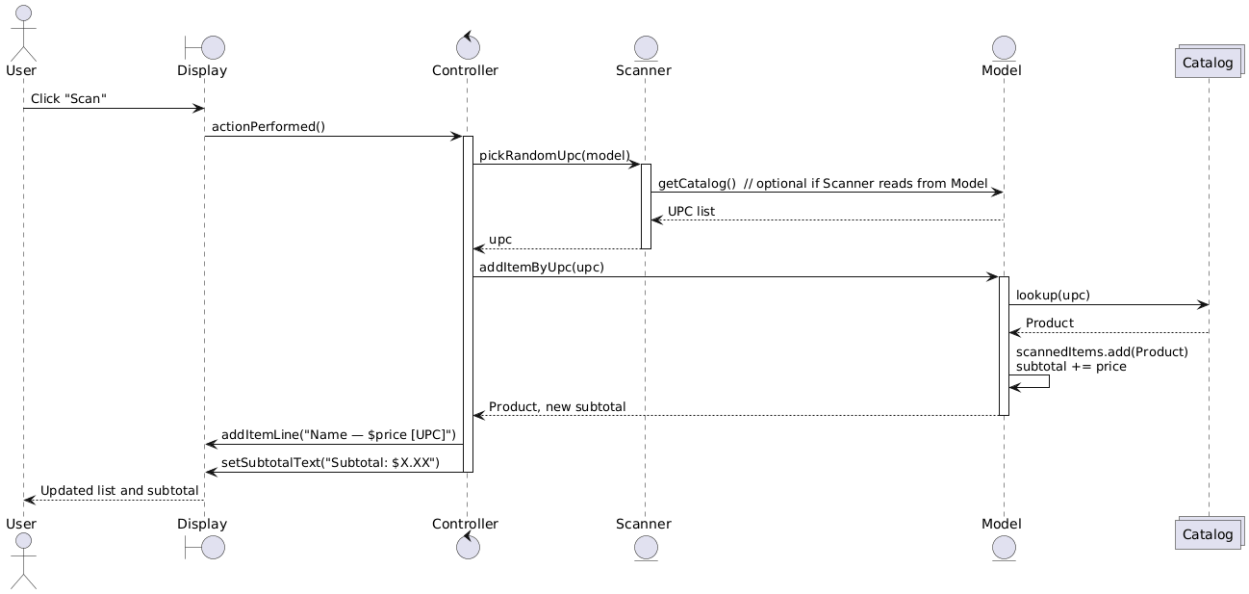
        setSize(520, 420);
        setLocationRelativeTo(null);
    }

    public JButton getScanButton() { return scanButton; }
    public JButton getClearButton() { return clearButton; }
    public JButton getAddByUpcButton() { return addByUpcButton; }
    public String getEnteredUpc() { return upcField.getText().trim(); }
    public void clearUpcField() { upcField.setText(""); }

    public void addItemLine(String line) {
        listModel.addElement(line);
        itemList.ensureIndexIsVisible(listModel.size() - 1);
    }

    public void resetList() { listModel.clear(); }
    public void setSubtotalText(String text) { subtotalLabel.setText(text); }
}
```

Part 4


User
Display
Controller
Scanner
Model
Catalog
Click "Scan"
actionPerformed()
pickRandomUpc(model)
getCatalog() // optional if Scanner reads from Model
UPC list
upc
addItemByUpc(upc)
lookup(upc)
Product
scannedItems.add(Product)
subtotal += price
Product, new subtotal
addItemLine("Name — $price [UPC]")
setSubtotalText("Subtotal: $X.XX")
Updated list and subtotal
User
Display
Controller
Scanner
Model
Catalog